

Real_data_garchx

Qiyang Wang

2026-04-19

Contents

1	Function preparation: auto_garchx and SynthVolForecast	1
1.1	Auto-garch	1
1.2	SynthVolForecast	4
2	simulation experiment	11
3	real data experiment	13
3.1	Theoretical Basis for the Real-Data Experiment	13
3.2	Candidate Donor Pools and Software Workflow	14
3.3	Comparison Across Donor-Pool Specifications	24

```
library(postshock)
```

```
## Registered S3 method overwritten by 'quantmod':  
##   method      from  
##   as.zoo.data.frame zoo
```

1 Function preparation: auto_garchx and SynthVolForecast

1.1 Auto-garch

```
auto_garchx <- function(  
  y,  
  xreg      = NULL,  
  max.p     = 3,  
  max.q     = 3,  
  search_o  = c(0L, 1L),      # 0 = symmetric, 1 = GJR  
  final_refit = TRUE,
```

```

    clamp_factor = c(0.1, 10),
    vcov.type     = c("ordinary", "robust"),
    verbose       = TRUE
) {
  vcov.type <- match.arg(vcov.type)

  # ---- Basic input checks -----
  stopifnot(is.numeric(y), is.vector(y))
  if (any(!is.finite(y))) stop("auto_garchx: 'y' contains NA/Inf/NaN.")
  n <- length(y)
  if (n < 20 && verbose) warning("auto_garchx: very short series; estimation may be unstable.")

  if (!is.null(xreg)) {
    xreg <- as.matrix(xreg)
    if (nrow(xreg) != n) stop("auto_garchx: nrow(xreg) must equal length(y).")
    if (any(!is.finite(xreg))) stop("auto_garchx: 'xreg' contains NA/Inf/NaN.")
  }

  # Normalize search list for o (0 or 1 only)
  search_o <- as.integer(unique(search_o))
  search_o <- search_o[search_o %in% c(0L, 1L)]
  if (!length(search_o)) search_o <- 0L

  # Holder for the best model by BIC
  best <- list(bic = Inf, aic = Inf, fit = NULL,
              p = NA_integer_, o = NA_integer_, q = NA_integer_)

  # ---- 1) Grid search over (q, p, o) -----
  # NOTE: garchx::garchx(order = c(q, p, o))
  for (q in 0:max.q) {
    for (p in 0:max.p) {
      for (o in search_o) {
        # skip the trivial model
        if (p == 0L && q == 0L && o == 0L) next

        fit0 <- try(
          garchx::garchx(y = y, order = c(q, p, o), xreg = xreg, vcov.type = vcov.type),
          silent = TRUE
        )
        if (inherits(fit0, "try-error")) next

        ll <- as.numeric(logLik(fit0))
        k <- length(coef(fit0))
        bic0 <- -2 * ll + k * log(n)
        aic0 <- -2 * ll + 2 * k

        if (is.finite(bic0) && bic0 < best$bic) {

```

```

    best <- list(bic = bic0, aic = aic0, fit = fit0, p = p, o = o, q = q)
  }
}
}
}
if (is.null(best$fit)) stop("auto_garchx: no converged model in grid search.")

# If we don't want the refit step, return the grid winner
if (!isTRUE(final_refit)) return(best)

# ---- 2) Refit using a backcast at unconditional variance -----
# Extract coefficients from the grid winner to compute unconditional variance
cf <- coef(best$fit)
om.idx <- grep("(intercept|omega)$", names(cf), ignore.case = TRUE)
if (!length(om.idx)) return(best) # if we cannot detect omega, give up refit

omega <- as.numeric(cf[om.idx[1]])
alpha <- sum(cf[grep("^arch", names(cf), ignore.case = TRUE)], na.rm = TRUE)
beta <- sum(cf[grep("^garch", names(cf), ignore.case = TRUE)], na.rm = TRUE)
gamma <- if (best$o == 1L) sum(cf[grep("asym|gamma|gjr", names(cf), ignore.case = TRUE)], na.rm = TRUE)

# Unconditional variance for GJR: omega / (1 - alpha - beta - 0.5*gamma)
denom <- 1 - alpha - beta - 0.5 * gamma
if (!is.finite(omega) || denom <= 0) return(best) # no valid refit

uncond_var <- as.numeric(omega / denom)

# Clamp to a reasonable range relative to sample variance
sv <- stats::var(y, na.rm = TRUE)
rng <- range(clamp_factor)
if (!is.finite(uncond_var) || uncond_var <= 0 ||
    uncond_var < rng[1] * sv || uncond_var > rng[2] * sv) {
  if (verbose) warning("[auto_garchx] backcast out of range; fallback to sample variance.")
  uncond_var <- sv
}
if (verbose) {
  message(sprintf("[auto_garchx] refit with backcast=%.6f (q=%d,p=%d,o=%d)",
    uncond_var, best$q, best$p, best$o))
}

# Refit with backcast.values set to a single numeric (not a list)
fit2 <- try(
  garchx::garchx(
    y = y,
    order = c(best$q, best$p, best$o),
    xreg = xreg,
    backcast.values = uncond_var,

```

```

    vcov.type = vcov.type
  ),
  silent = TRUE
)

# If refit succeeded, keep it only if BIC improves
if (!inherits(fit2, "try-error")) {
  ll2 <- as.numeric(logLik(fit2))
  k2 <- length(coef(fit2))
  bic2 <- -2 * ll2 + k2 * log(n)
  aic2 <- -2 * ll2 + 2 * k2

  if (is.finite(bic2) && bic2 < best$bic) {
    best$fit <- fit2
    best$bic <- bic2
    best$aic <- aic2
    best$refit_used <- TRUE
  } else {
    best$refit_used <- FALSE
    if (verbose) warning("[auto_garchx] refit did not improve BIC; keeping grid-search model")
  }
} else if (verbose) {
  warning("[auto_garchx] backcast-refit failed; keeping grid-search model.")
}

best
}

#' @rdname auto_garchx
#' @export
auto_garch <- auto_garchx

```

1.2 SynthVolForecast

```

SynthVolForecast <- function(
  Y_series_list,
  covariates_series_list,
  target_covariates,
  shock_time_vec,
  shock_length_vec,
  k = 1,
  dbw_scale = TRUE,
  dbw_center = TRUE,
  dbw_indices = NULL,
  princ_comp_input = NULL,

```

```

    covariate_indices = NULL,
    penalty_lambda = 0,
    penalty_normchoice = "l1",
    plots = TRUE,
    shock_time_labels = NULL,
    return_fits = FALSE,
    garch_order = NULL,
    max.p = 3,
    max.q = 3,
    backcast.initial = NULL,
    vcov.type = "robust",
    target_use_xreg = FALSE,
    target_future_xreg = NULL
) {
  # ---- sanitize args -----
  k <- max(1L, as.integer(k))
  vc <- match.arg(tolower(as.character(vcov.type)), c("ordinary", "robust"))

  n_total <- length(Y_series_list)
  n_donors <- n_total - 1L

  if (!vc %in% c("ordinary", "robust"))
    stop("vcov.type must be 'ordinary' or 'robust'.")
  if (!is.null(backcast.initial)) {
    stopifnot(is.numeric(backcast.initial), length(backcast.initial) == 1,
              is.finite(backcast.initial), backcast.initial > 0)
  }

  # ---- FAST PATH: no donors -----
  if (n_donors == 0L) {
    target_Y <- Y_series_list[[1L]]
    target_end <- as.integer(shock_time_vec[1L])

    if (isTRUE(target_use_xreg) && !is.null(covariate_indices)) {
      X_target <- as.matrix(target_covariates[1:target_end, covariate_indices, drop = FALSE])

      # lag 1
      X_target_lag <- rbind(X_target[1, , drop = FALSE], X_target[-nrow(X_target), , drop = FALSE])
      X_target_lag[1, ] <- 0

      target_xreg <- X_target_lag

      # future xreg- last observation carried forward
      if (is.null(target_future_xreg)) {
        X_last <- as.matrix(target_covariates[target_end, covariate_indices, drop = FALSE])
        target_future_xreg <- matrix(
          rep(as.numeric(X_last), k),

```

```

    nrow = k,
    byrow = TRUE
  )
  colnames(target_future_xreg) <- colnames(X_target_lag)
} else {
  target_future_xreg <- as.matrix(target_future_xreg)
}
} else {
  target_xreg <- NULL
  target_future_xreg <- NULL
}

if (is.null(garch_order)) {
  sel_t <- auto_garchx(
    y = target_Y[1:target_end], xreg = target_xreg,
    max.p = max.p, max.q = max.q,
    vcov.type = vc
  )
  target_order <- c(sel_t$q, sel_t$p, sel_t$o)
  fit_target <- sel_t$fit
} else {
  target_order <- garch_order
  fit_target <- garchx::garchx(
    y = target_Y[1:target_end],
    order = target_order,
    xreg = target_xreg,
    vcov.type = vc,
    backcast.values = if (is.null(backcast.initial)) NULL else backcast.initial
  )
}

raw_pred <- if (is.null(target_future_xreg)) {
  stats::predict(fit_target, n.ahead = k)
} else {
  stats::predict(fit_target, n.ahead = k, newxreg = target_future_xreg)
}

raw_vec <- as.numeric(if (is.list(raw_pred)) raw_pred$pred else raw_pred)

return(list(
  linear_combinations = numeric(0),
  predictions = list(
    unadjusted = raw_vec,
    adjusted = raw_vec,
    arithmetic_mean = raw_vec
  ),
  meta = list(
    n_donors = 0L,

```

```

    weights      = numeric(0),
    omega_vec     = numeric(0),
    omega_se      = numeric(0),
    combined_omega = 0,
    shock_time    = shock_time_vec,
    shock_length  = shock_length_vec,
    target_order  = target_order
  ),
  target_fit = fit_target
))
}

# ---- donor path checks -----
stopifnot(length(shock_time_vec) == n_total)
stopifnot(length(shock_length_vec) == n_total)
stopifnot(length(covariates_series_list) == n_donors)

# helper to coerce to data.frame
as_df <- function(x) {
  if (is.numeric(x)) data.frame(V1 = x)
  else if (is.matrix(x)) as.data.frame(x)
  else if (inherits(x, "data.frame")) x
  else stop("covariates must be numeric/matrix/data.frame")
}

target_covariates <- as_df(target_covariates)
for (i in seq_len(n_donors)) {
  covariates_series_list[[i]] <- as_df(covariates_series_list[[i]])
}

if (is.null(dbw_indices)) dbw_indices <- seq_len(ncol(target_covariates))
if (is.null(princ_comp_input)) princ_comp_input <- min(length(shock_time_vec),
  ncol(target_covariates))

has_lmtest <- requireNamespace("lmtest", quietly = TRUE)

# ---- per-donor fit and omega extraction -----
omega_star_hat_vec <- numeric(n_donors)
omega_star_std_err_vec <- rep(NA_real_, n_donors)
donor_fits <- vector("list", n_donors)
donor_selected_orders <- vector("list", n_donors)

for (i in seq_len(n_donors)) {
  donor_Y <- Y_series_list[[i+1L]]
  donor_X <- covariates_series_list[[i]]

  len_i <- as.integer(shock_length_vec[i+1L])

```

```

start_i <- as.integer(shock_time_vec[i+1L])
end_i <- min(start_i + len_i, length(donor_Y))
post_shock_indicator <- rep(0L, length(donor_Y))
if (len_i > 0L && end_i >= start_i + 1L) {
  post_shock_indicator[(start_i+1L):end_i] <- 1L
}

rows_to_use <- length(donor_Y)

if (is.null(covariate_indices)) {
  X_i_final <- matrix(
    post_shock_indicator[1:rows_to_use],
    ncol = 1,
    dimnames = list(NULL, "post_shock_indicator")
  )
  donor_y_final <- donor_Y[1:rows_to_use]
} else {
  X_cov <- as.matrix(donor_X[1:rows_to_use, covariate_indices, drop = FALSE])
  X_i_final <- cbind(X_cov, post_shock_indicator[1:rows_to_use])
  colnames(X_i_final)[ncol(X_i_final)] <- "post_shock_indicator"
  X_i_final <- stats::na.omit(X_i_final)

  donor_y_final <- donor_Y[1:nrow(X_i_final)]
}

if (is.null(garch_order)) {
  sel <- auto_garchx(
    y = donor_y_final,
    xreg = X_i_final,
    max.p = max.p,
    max.q = max.q,
    vcov.type = vc
  )
  order_i <- c(sel$q, sel$p, sel$o)
  donor_fit <- sel$fit
} else {
  order_i <- garch_order
  donor_fit <- garchx::garchx(
    y = donor_y_final,
    order = order_i,
    xreg = X_i_final,
    vcov.type = vc,
    backcast.values = if (is.null(backcast.initial)) NULL else backcast.initial
  )
}

donor_selected_orders[[i]] <- order_i

```

```

donor_fits[[i]] <- donor_fit

# omega = coefficient on the post-shock indicator
if (has_lmtest) {
  vcov_fun <- function(obj) stats::vcov(obj, vcov.type = vc)
  ct <- lmtest::coeftest(donor_fit, vcov. = vcov_fun)
  hit <- grep("post_shock_indicator", rownames(ct), fixed = TRUE)
  omega_star_hat_vec[i] <- ct[hit[1L], "Estimate"]
  if ("Std. Error" %in% colnames(ct)) {
    omega_star_std_err_vec[i] <- ct[hit[1L], "Std. Error"]
  }
} else {
  cf <- coef(donor_fit)
  nm <- names(cf)
  hit <- grep("post_shock_indicator", nm, fixed = TRUE)
  stopifnot(length(hit) >= 1L)
  omega_star_hat_vec[i] <- cf[hit[1L]]
}
}

# ---- DBW weights (target + donors) -----
X_for_dbw <- c(list(target_covariates), covariates_series_list)
dbw_output <- dbw(
  X = X_for_dbw,
  dbw_indices = dbw_indices,
  shock_time_vec = shock_time_vec,
  scale = dbw_scale,
  center = dbw_center,
  sum_to_1 = TRUE,
  bounded_below_by = 0,
  bounded_above_by = 1,
  normchoice = "l2",
  penalty_normchoice = penalty_normchoice,
  penalty_lambda = penalty_lambda
)
w_hat <- as.numeric(dbw_output$opt_params)
stopifnot(length(w_hat) == length(omega_star_hat_vec))
omega_star_hat <- sum(w_hat * omega_star_hat_vec)

# ---- target pre-shock model and forecast -----
target_Y <- Y_series_list[[1L]]
target_end <- as.integer(shock_time_vec[1L])

# Target NEVER uses xreg in this function (avoid needing future newxreg)
target_xreg <- NULL

if (is.null(garch_order)) {

```

```

sel_t <- auto_garchx(
  y      = target_Y[1:target_end],
  xreg   = target_xreg,
  max.p  = max.p,
  max.q  = max.q,
  vcov.type = vc
)
target_order <- c(sel_t$q, sel_t$p, sel_t$o)
fit_target   <- sel_t$fit
} else {
  target_order <- garch_order
  fit_target   <- garchx::garchx(
    y = target_Y[1:target_end],
    order = target_order,
    xreg = target_xreg,
    vcov.type = vc,
    backcast.values = if (is.null(backcast.initial)) NULL else backcast.initial
  )
}

raw_pred <- stats::predict(fit_target, n.ahead = k)
raw_vec  <- as.numeric(if (is.list(raw_pred)) raw_pred$pred else raw_pred)

eps <- 1e-8
adjusted_vec <- pmax(raw_vec + omega_star_hat, eps)
arithmetic_vec <- pmax(raw_vec + mean(omega_star_hat_vec), eps)

predictions <- list(
  unadjusted      = raw_vec,
  adjusted        = adjusted_vec,
  arithmetic_mean = arithmetic_vec
)

meta <- list(
  n_donors      = n_donors,
  shock_time    = shock_time_vec,
  shock_length  = shock_length_vec,
  dbw_status    = dbw_output$convergence,
  weights       = w_hat,
  omega_vec     = omega_star_hat_vec,
  omega_se      = omega_star_std_err_vec,
  combined_omega = omega_star_hat,
  donor_orders  = donor_selected_orders,
  target_order  = target_order
)

```

```

out <- list(
  linear_combinations = w_hat,
  predictions          = predictions,
  meta                 = meta
)
if (return_fits) {
  out$donor_fits <- donor_fits
  out$target_fit <- fit_target
}

# ---- optional plot hook -----
if (isTRUE(plots)) {
  f <- get0("plot_maker_garch", mode = "function", inherits = TRUE)
  if (!is.null(f)) {
    try(
      f(
        stats::fitted(fit_target),
        shock_time_labels,
        shock_time_vec,
        shock_length_vec,
        raw_vec,
        w_hat,
        omega_star_hat_vec,
        omega_star_std_err_vec,
        adjusted_vec,
        arithmetic_vec
      ),
      silent = TRUE
    )
  }
}

out
}

```

2 simulation experiment

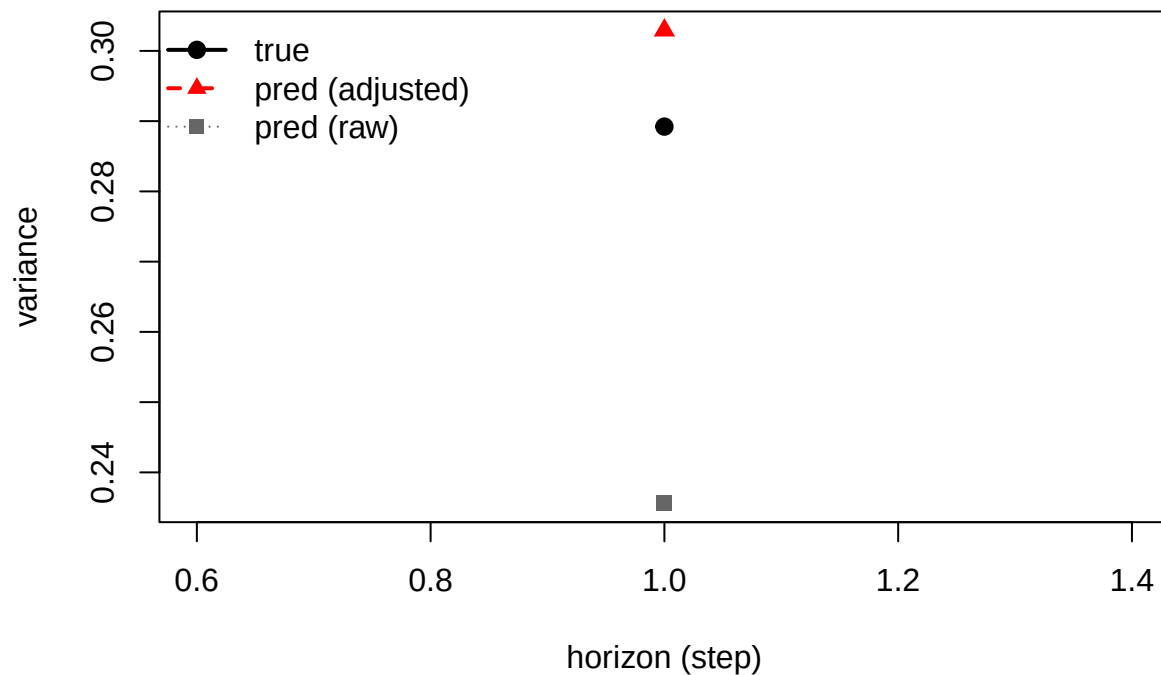
```
## === omega-jump: single-origin testing (k = 1-2) ===
```

```
##
## Iter: 1 fn: 0.9311    Pars:  0.98134128120 0.00000007001 0.01865864879
## Iter: 2 fn: 0.9311    Pars:  0.98134129705 0.00000003564 0.01865866731
## Iter: 3 fn: 0.9311    Pars:  0.98134130308 0.00000002346 0.01865867346
## solnp--> Completed in 3 iterations
```

```
##  step true_var pred_raw pred_adj diff_adj abs_pct_err_adj
```

```
##      1 0.289223 0.235606 0.302921 0.013698      4.736135
##
## [k=1] MSE(true, adj)=0.000188 | MAE=0.013698 | Corr=NA
## weights (w_hat): 0.981, 0, 0.019
## omega by donor : 0.068, 0.082, 0.026
## combined omega : 0.067
```

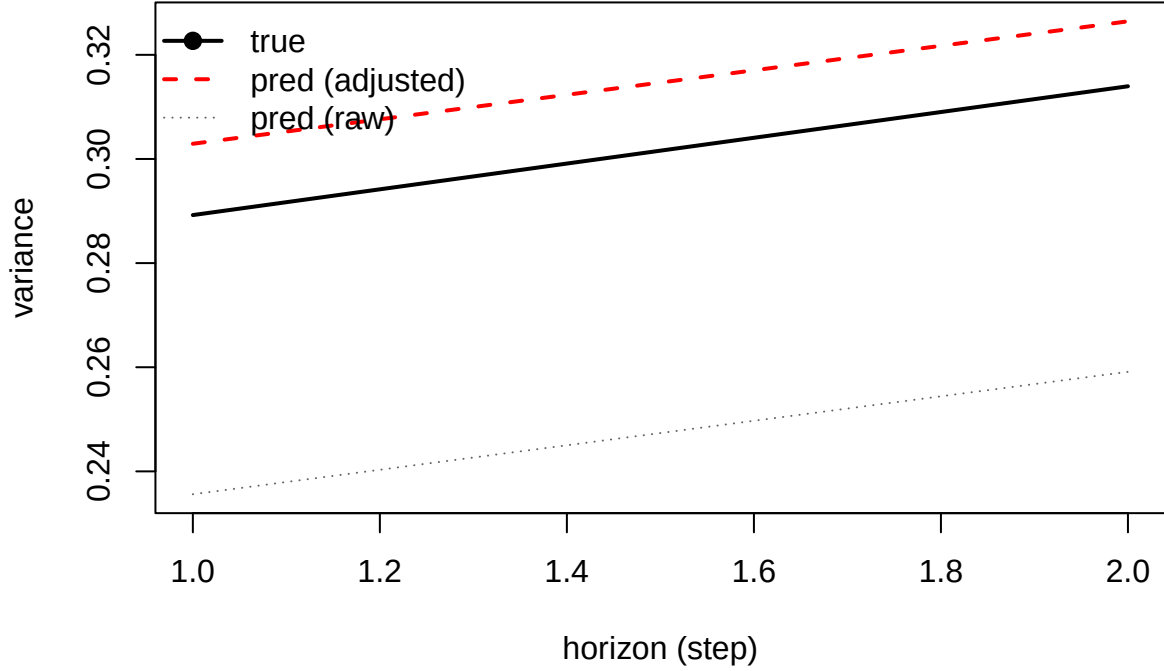
True vs Predicted variance (k=1)



```
##
## Iter: 1 fn: 0.9311 Pars: 0.98134128120 0.00000007001 0.01865864879
## Iter: 2 fn: 0.9311 Pars: 0.98134129705 0.00000003564 0.01865866731
## Iter: 3 fn: 0.9311 Pars: 0.98134130308 0.00000002346 0.01865867346
## solnp--> Completed in 3 iterations
```

```
## step true_var pred_raw pred_adj diff_adj abs_pct_err_adj
##      1 0.289223 0.235606 0.302921 0.013698      4.736135
##      2 0.313962 0.259117 0.326432 0.012470      3.971863
##
## [k=2] MSE(true, adj)=0.000172 | MAE=0.013084 | Corr=1.000
## weights (w_hat): 0.981, 0, 0.019
## omega by donor : 0.068, 0.082, 0.026
## combined omega : 0.067
```

True vs Predicted variance (k=2)



3 real data experiment

3.1 Theoretical Basis for the Real-Data Experiment

Our real-data experiment focuses on **post-shock volatility forecasting** rather than mean forecasting. The key idea is that GARCH-type models do not model price levels directly; instead, they model the **time-varying conditional variance** of a return or innovation process.

Let

$$r_t = \mu_t + \epsilon_t, \quad \epsilon_t = \sigma_t \eta_t,$$

where r_t is the observed return, μ_t is the conditional mean, ϵ_t is the innovation, and σ_t^2 is the conditional variance. In this framework, GARCH models the dynamics of σ_t^2 , not the mean itself. Thus, the usual input to GARCH is a **return series** or a **mean-corrected residual series**, while the object of interest is its volatility.

This also explains the relation between ARIMA and GARCH. ARIMA mainly describes the **conditional mean structure**, while GARCH adds a second layer to describe the **conditional variance structure** of the innovation process. For financial returns, the mean is often weak, while volatility clustering is strong, making GARCH-type modeling especially suitable.

GARCHX extends GARCH by allowing exogenous variables to enter the variance equation:

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2 + \lambda^\top x_t.$$

Here, x_t may include market covariates or shock-related variables. Therefore, GARCHX is appropriate when post-shock volatility may be affected not only by lagged shocks and lagged variance, but also by external information such as uncertainty measures, interest rates, or a post-shock indicator.

In our setting, the target series is a return-like financial series, so using daily log returns as the GARCHX input is theoretically appropriate. The model is then used to forecast short-horizon post-shock volatility.

Our current **SynthVolForecast** framework adds a donor-based adjustment on top of the target-side volatility forecast. The logic is as follows:

1. Fit donor episodes with a GARCHX specification containing a post-shock indicator.
2. Extract donor-specific post-shock effects.
3. Combine these effects using donor-basis weights.
4. Add the weighted shock effect to the target’s baseline volatility forecast.

This design is especially reasonable because our goal is to forecast the **first one or two days after the shock**, rather than the full long-run recovery path. Therefore, the donor-weighted additive shock correction should be interpreted as a **short-horizon post-shock volatility uplift adjustment**.

Finally, in real data, the true conditional variance is latent and cannot be directly observed. As a result, the model input and the forecast evaluation target must be distinguished: returns or residuals are used for modeling, while volatility proxies are used for empirical evaluation.

3.2 Candidate Donor Pools and Software Workflow

To make the donor-pool design transparent, we treat donor-pool choice as an analyst-specified input rather than something determined by ex post forecast performance. The target episode in this application is the 2016 Election. Given a specified donor pool, our software can quickly return the corresponding post-shock volatility forecast, which makes it straightforward to compare outputs across competing donor-pool specifications within a common framework. In this application, we consider several substantively motivated donor-pool constructions based on earlier political and macro-uncertainty events that may be relevant to short-horizon post-shock volatility in the financial sector.

3.2.1 Full Candidate Donor Pool

In this application, the full candidate donor pool includes the 2004 Election, 2008 Election, 2012 Election, Brexit Announced, Brexit Poll Released, and Brexit. This provides the broadest donor-pool specification considered in the empirical comparison.

Given this full donor-pool specification, the corresponding target and donor inputs can be constructed directly from the episode panels and passed into the common forecasting workflow.

```
# =====
# Step 1: define episode metadata
# =====
```

```

target_symbol <- "IYG"
pre_n_default <- 500
post_buffer_default <- 10

episode_meta <- data.frame(
  episode_name = c(
    "2016 Election",
    "Brexit",
    "Brexit Poll Released",
    "Brexit Announced",
    "2012 Election",
    "2004 Election",
    "2008 Election"
  ),
  shock_date = as.Date(c(
    "2016-11-08",
    "2016-06-23",
    "2016-06-13",
    "2016-02-19",
    "2012-11-06",
    "2004-11-02",
    "2008-11-04"
  )),
  asset = target_symbol,
  pre_n = pre_n_default,
  post_buffer = post_buffer_default
)

episode_meta <- episode_meta[order(episode_meta$shock_date), ]
rownames(episode_meta) <- NULL

print(episode_meta)

```

```

##           episode_name shock_date asset pre_n post_buffer
## 1      2004 Election 2004-11-02   IYG   500          10
## 2      2008 Election 2008-11-04   IYG   500          10
## 3      2012 Election 2012-11-06   IYG   500          10
## 4    Brexit Announced 2016-02-19   IYG   500          10
## 5 Brexit Poll Released 2016-06-13   IYG   500          10
## 6              Brexit 2016-06-23   IYG   500          10
## 7      2016 Election 2016-11-08   IYG   500          10

## [1] "2004 Election"      "2008 Election"      "2012 Election"
## [4] "Brexit Announced"   "Brexit Poll Released" "Brexit"
## [7] "2016 Election"

##           2004 Election      2008 Election      2012 Election

```

```
##          511          511          511
##   Brexit Announced Brexit Poll Released   Brexit
##          511          511          511
##   2016 Election
##          511
```

```
##   2004 Election   2008 Election   2012 Election
##   "2004-11-02"   "2008-11-04"   "2012-11-06"
##   Brexit Announced Brexit Poll Released   Brexit
##   "2016-02-19"   "2016-06-13"   "2016-06-23"
##   2016 Election
##   "2016-11-08"
```

```
##          episode_name shock_date start_date  end_date
## 2004 Election          2004 Election 2004-11-02 2002-10-08 2004-11-16
## 2008 Election          2008 Election 2008-11-04 2006-10-13 2008-11-18
## 2012 Election          2012 Election 2012-11-06 2010-10-15 2012-11-20
## Brexit Announced      Brexit Announced 2016-02-19 2014-01-28 2016-03-04
## Brexit Poll Released  Brexit Poll Released 2016-06-13 2014-05-21 2016-06-27
## Brexit                  Brexit 2016-06-23 2014-06-04 2016-07-11
## 2016 Election          2016 Election 2016-11-08 2014-10-17 2016-11-28
##          shock_index_in_panel n_rows
## 2004 Election          501    511
## 2008 Election          501    511
## 2012 Election          501    511
## Brexit Announced      501    511
## Brexit Poll Released    501    511
## Brexit                  501    511
## 2016 Election          501    511
```

```
##          ret_IYG      ret_CLF VIX_level  dIRX      dTNX
## 2014-10-17  0.012197632  0.0006044491    21.99  0.002  0.045999765
## 2014-10-20  0.005739223 -0.0004835116    18.57 -0.002 -0.017999887
## 2014-10-21  0.020606789  0.0012082949    16.08 -0.002  0.026999950
## 2014-10-22 -0.010913113 -0.0280432385    17.87 -0.003  0.021000147
## 2014-10-23  0.011151982  0.0193106027    16.53  0.000  0.046000004
## 2014-10-24  0.010320559 -0.0132435315    16.11 -0.010 -0.002000093
```

```
##          ret_IYG      ret_CLF VIX_level      dIRX      dTNX
## 2016-11-17  0.0130431788 -0.003297102    13.35 -0.033000022  0.052999973
## 2016-11-18  0.0008898021  0.005926929    12.85 -0.001999974  0.059000015
## 2016-11-21  0.0037487118  0.038639772    12.42  0.001999974  0.003999949
## 2016-11-22  0.0004923123  0.011306592    12.41  0.055000007 -0.017999887
## 2016-11-23  0.0048107287 -0.001458479    12.43  0.010000020  0.036000013
## 2016-11-28 -0.0154462595  0.021903397    13.15 -0.030000001 -0.052000046
```

```
## [1] 511    5
```

```
## [1] "ret_IYG"    "ret_CLF"    "VIX_level" "dIRX"      "dTNX"
```

As a preliminary descriptive diagnostic, we inspected the local $[-5, +5]$ window around each candidate episode and examined the timing of the largest absolute return. This exercise was used only as a plausibility check on local event dynamics and not as a formal donor-selection rule. The restricted donor set is instead motivated by ex ante comparability in event type, economic interpretation, and financial-sector relevance.

3.2.2 Full Donor-Pool Forecast Result

The full donor pool serves as a broad benchmark specification. Its main advantage is that it avoids excessive screening and allows the donor-adjustment procedure to incorporate a wider range of historical episodes. This specification is useful for robustness, since it reduces the risk that the reported results are driven by a narrowly selected donor set.

```
svf_out_all <- SynthVolForecast(
  Y_series_list = Y_series_list,
  covariates_series_list = covariates_series_list,
  target_covariates = target_covariates,
  shock_time_vec = shock_time_vec,
  shock_length_vec = shock_length_vec,
  k = 1,
  dbw_indices = 1:ncol(target_covariates),
  covariate_indices = NULL,
  plots = FALSE,
  return_fits = TRUE,
  max.p = 3,
  max.q = 3,
  vcov.type = "robust"
)
```

```
##
## Iter: 1 fn: 0.2445    Pars:  0.41066380285 0.00000003081 0.00000014390 0.02669045149 0.0000
## Iter: 2 fn: 0.2445    Pars:  0.4106638302380 0.0000000013258 0.0000000716397 0.026690450787
## Iter: 3 fn: 0.2445    Pars:  4.107e-01 1.195e-09 7.094e-08 2.669e-02 1.124e-11 5.626e-01
## solnp--> Completed in 3 iterations
```

```
svf_out_all$predictions
```

```
## $unadjusted
## [1] 6.782242e-05
##
## $adjusted
## [1] 0.0007783468
##
## $arithmetic_mean
## [1] 0.0008967612
```

```
svf_out_all$meta$weights
```

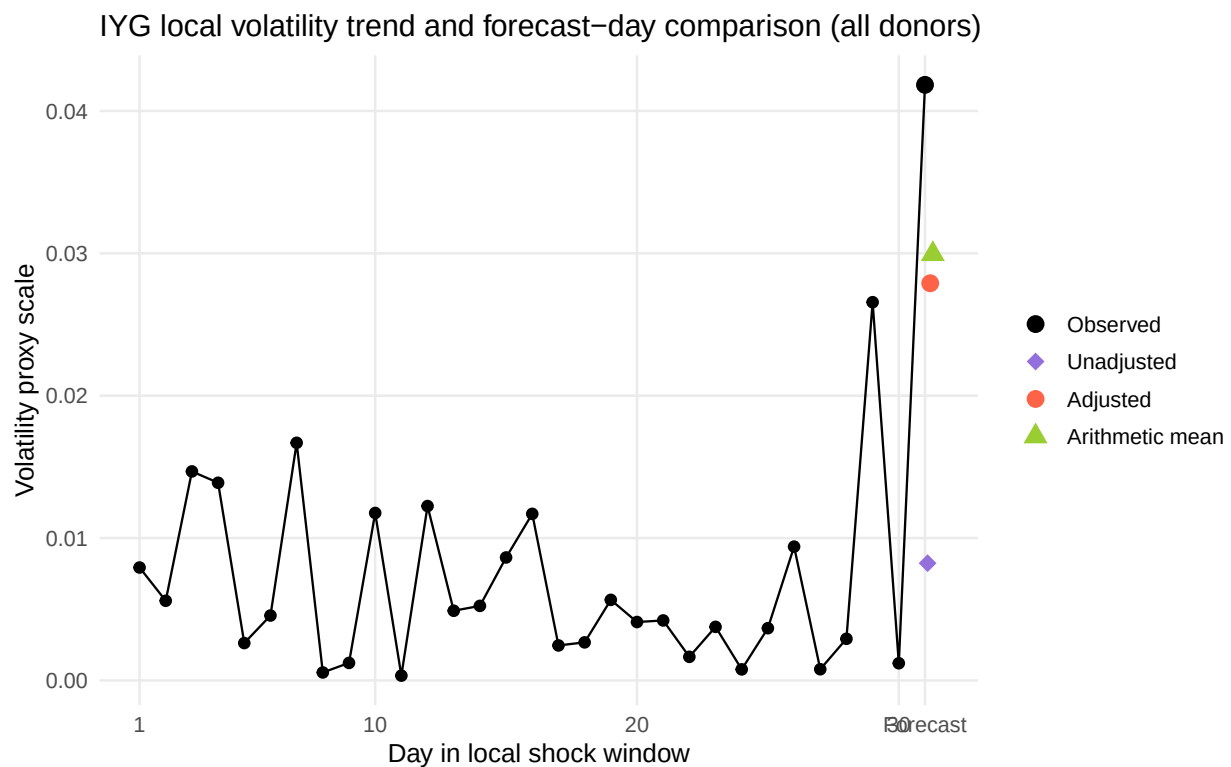
```
## [1] 4.106638e-01 1.195493e-09 7.093793e-08 2.669045e-02 1.123796e-11  
## [6] 5.626456e-01
```

```
svf_out_all$meta$omega_vec
```

```
## [1] 0.0000000000 0.0010580385 0.0002876665 0.0001895676 0.0021845254  
## [6] 0.0012538349
```

```
Y_series_list[[1]][502]^2
```

```
## [1] 0.001750284
```



Under the full donor-pool specification, the software returns a donor-adjusted one-step-ahead volatility forecast that is substantially above the no-donor baseline forecast. In this episode, the target-only baseline forecast is very small relative to the observed next-day variance proxy, whereas both the donor-adjusted forecast and the arithmetic-mean benchmark imply a much larger short-horizon post-shock volatility response. This indicates that, even under a broad donor pool, donor information continues to suggest a sizable volatility uplift beyond the target-side pre-shock forecast.

A donor-level inspection suggests that the full-pool adjustment is not driven uniformly by all election episodes. In particular, the 2004 Election receives substantial weight but contributes little because its estimated donor-side shock effect is close to zero, while Brexit contributes most strongly because it combines a large weight with a nontrivial estimated shock effect.

3.2.3 Restricted Donor Set

A second analyst choice is to work with a more focused donor pool based on ex ante comparability. In this application, we consider a restricted donor set consisting of the 2012 Election, Brexit Poll Released, and Brexit. This restriction is not based on ex post forecast performance or on whether donor-side estimated shock effects are statistically significant. Rather, it reflects a substantive judgment that these episodes provide a more focused set of historical analogs for the 2016 Election. The 2012 Election is the closest event in type to the target episode, while Brexit Poll Released and Brexit capture discrete political uncertainty shocks with plausible spillovers to global risk sentiment and the financial sector. Given this restricted donor-pool specification, the corresponding target and donor inputs can be constructed directly from the episode panels and passed into the common forecasting workflow.

3.2.4 Restricted Donor-Pool Forecast Result

```
svf_out_k1 <- SynthVolForecast(  
  Y_series_list = Y_series_list,  
  covariates_series_list = covariates_series_list,  
  target_covariates = target_covariates,  
  shock_time_vec = shock_time_vec,  
  shock_length_vec = shock_length_vec,  
  k = 1,  
  dbw_indices = 1:ncol(target_covariates),  
  covariate_indices = NULL,  
  plots = FALSE,  
  return_fits = TRUE,  
  max.p = 3,  
  max.q = 3,  
  vcov.type = "robust"  
)  
  
## [auto_garchx] refit with backcast=0.000273 (q=1,p=0,o=1)  
  
## [auto_garchx] refit with backcast=0.000163 (q=1,p=0,o=1)  
  
## [auto_garchx] refit with backcast=0.000153 (q=1,p=0,o=1)  
  
##  
## Iter: 1 fn: 2.1308    Pars:  0.17069 0.38560 0.44372  
## Iter: 2 fn: 2.1308    Pars:  0.17071 0.38560 0.44368  
## solnp--> Completed in 2 iterations  
  
## [auto_garchx] refit with backcast=0.000198 (q=1,p=0,o=1)
```

```
target_reaction_proxy <- Y_series_list[[1]][502]^2
svf_out_k1$predictions$unadjusted
```

```
## [1] 6.782276e-05
```

```
svf_out_k1$predictions$adjusted
```

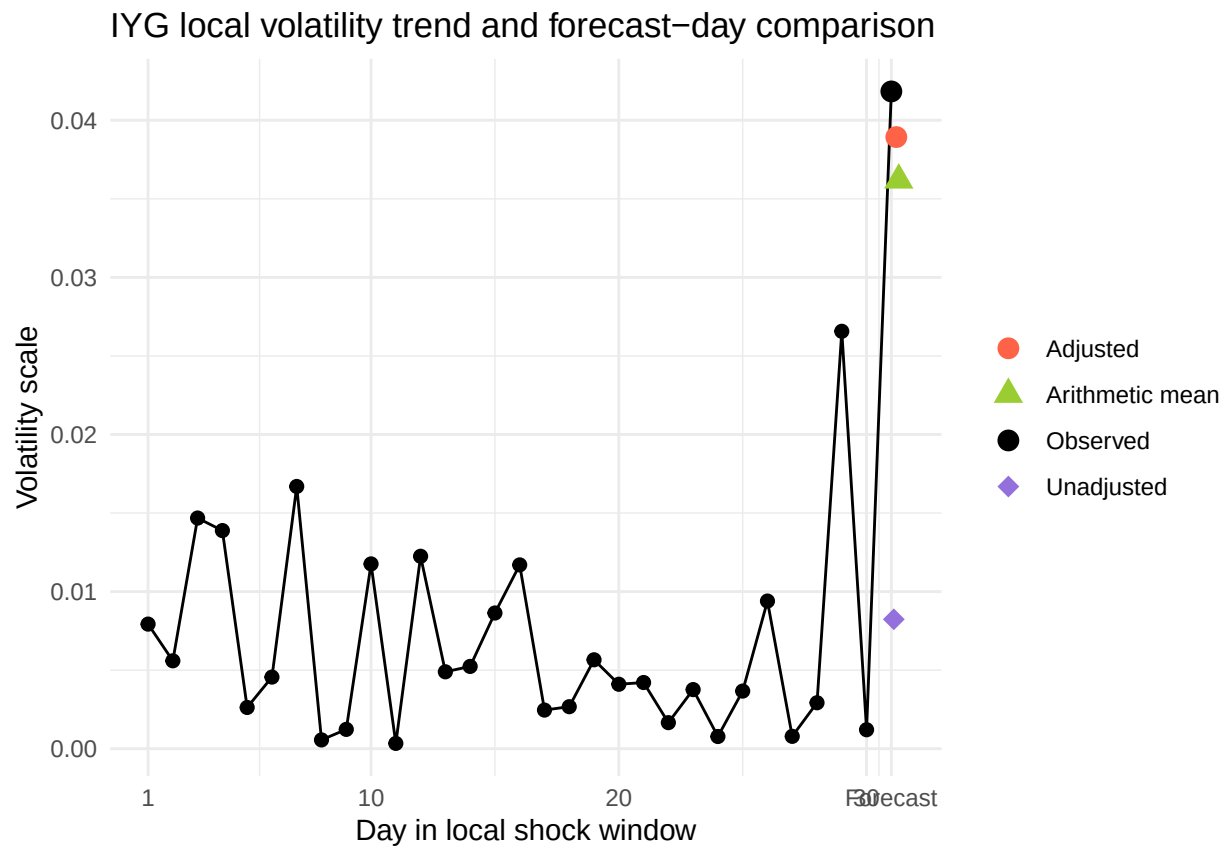
```
## [1] 0.00151559
```

```
svf_out_k1$predictions$arithmetic_mean
```

```
## [1] 0.001309829
```

```
target_reaction_proxy
```

```
## [1] 0.001750259
```



Under the restricted donor-pool specification, the software returns a donor-adjusted one-step-ahead volatility forecast that is substantially above the no-donor baseline forecast. In this episode, the donor-adjusted variance forecast is approximately 0.001516, compared with a no-donor baseline

forecast of about 0.000068. The arithmetic-mean benchmark is approximately 0.001310, while the observed next-day variance proxy is about 0.001750. Thus, both donor-based forecasts are substantially above the no-donor baseline, indicating that donor information under the restricted pool implies a sizable short-horizon post-shock volatility uplift. Because this donor pool is defined on ex ante comparability grounds, the resulting forecast should be interpreted as the output associated with a focused set of historical analogs rather than as a pool chosen on the basis of ex post performance.

3.2.5 Elections-Only Donor Pool

A third analyst choice is to use a donor pool consisting only of past U.S. presidential elections. In this application, we consider an elections-only donor pool made up of the 2004 Election, 2008 Election, and 2012 Election. This specification is also defined on ex ante grounds rather than on ex post forecast performance. Its main appeal is that all donor episodes are of the same event type as the 2016 Election target episode, which makes the pool especially transparent from an event-comparability perspective. At the same time, these elections occurred under different market environments, so this pool also provides a useful intermediate specification between the full candidate donor pool and the more focused restricted donor set. Given this elections-only donor-pool specification, the corresponding target and donor inputs can be constructed directly from the episode panels and passed into the common forecasting workflow.

```
# =====
# Elections-only donor pool
# =====

# define elections-only pool
event <- c(
  "2004 Election",
  "2008 Election",
  "2012 Election",
  "2016 Election"
)

target_event <- "2016 Election"
donor_events <- setdiff(event, target_event)

# helper: convert one episode panel to Y and X
extract_episode_inputs <- function(ep_obj) {
  panel_i <- ep_obj$panel

  Y_i <- as.numeric(panel_i[, "ret_IYG"])

  X_i <- data.frame(
    ret_CLF = as.numeric(panel_i[, "ret_CLF"]),
    VIX_level = as.numeric(panel_i[, "VIX_level"]),
    dIRX = as.numeric(panel_i[, "dIRX"]),
    dTNX = as.numeric(panel_i[, "dTNX"])
  )
}
```

```

)

list(Y = Y_i, X = X_i)
}

# target inputs
target_obj <- extract_episode_inputs(episode_data_list[[target_event]])

# donor inputs
donor_obj_list <- lapply(donor_events, function(nm) {
  extract_episode_inputs(episode_data_list[[nm]])
})
names(donor_obj_list) <- donor_events

# assemble SynthVolForecast inputs
Y_series_list <- c(
  list(target_obj$Y),
  lapply(donor_obj_list, function(x) x$Y)
)

target_covariates <- target_obj$X
covariates_series_list <- lapply(donor_obj_list, function(x) x$X)

# shock metadata
shock_time_vec <- rep(501, length(Y_series_list))
shock_length_vec <- rep(1, length(Y_series_list))

# fit elections-only specification
svf_out_elec <- SynthVolForecast(
  Y_series_list = Y_series_list,
  covariates_series_list = covariates_series_list,
  target_covariates = target_covariates,
  shock_time_vec = shock_time_vec,
  shock_length_vec = shock_length_vec,
  k = 1,
  dbw_indices = 1:ncol(target_covariates),
  covariate_indices = NULL,
  plots = FALSE,
  return_fits = TRUE,
  max.p = 3,
  max.q = 3,
  vcov.type = "robust"
)

```

```
## [auto_garchx] refit with backcast=0.000103 (q=1,p=1,o=0)
```

```
## Warning in auto_garchx(y = donor_y_final, xreg = X_i_final, max.p = max.p, :
```

```

## [auto_garchx] refit did not improve BIC; keeping grid-search model.

## Warning in sqrt(diag(se)): NaNs produced

## [auto_garchx] refit with backcast=0.000273 (q=1,p=0,o=1)

## Warning in sqrt(diag(se)): NaNs produced

## Warning in dbw(X = X_for_dbw, dbw_indices = dbw_indices, shock_time_vec =
## shock_time_vec, : Design matrix is rank-deficient (collinearity). Consider PCA.

##
## Iter: 1 fn: 0.3262 Pars: 0.425855648763 0.000000005008 0.574144346229
## Iter: 2 fn: 0.3262 Pars: 0.425855650107 0.000000002725 0.574144347168
## solnp--> Completed in 2 iterations

## [auto_garchx] refit with backcast=0.000198 (q=1,p=0,o=1)

target_reaction_proxy <- Y_series_list[[1]][502]^2

svf_out_elec$predictions$unadjusted

## [1] 6.782276e-05

svf_out_elec$predictions$adjusted

## [1] 0.0002329905

svf_out_elec$predictions$arithmetic_mean

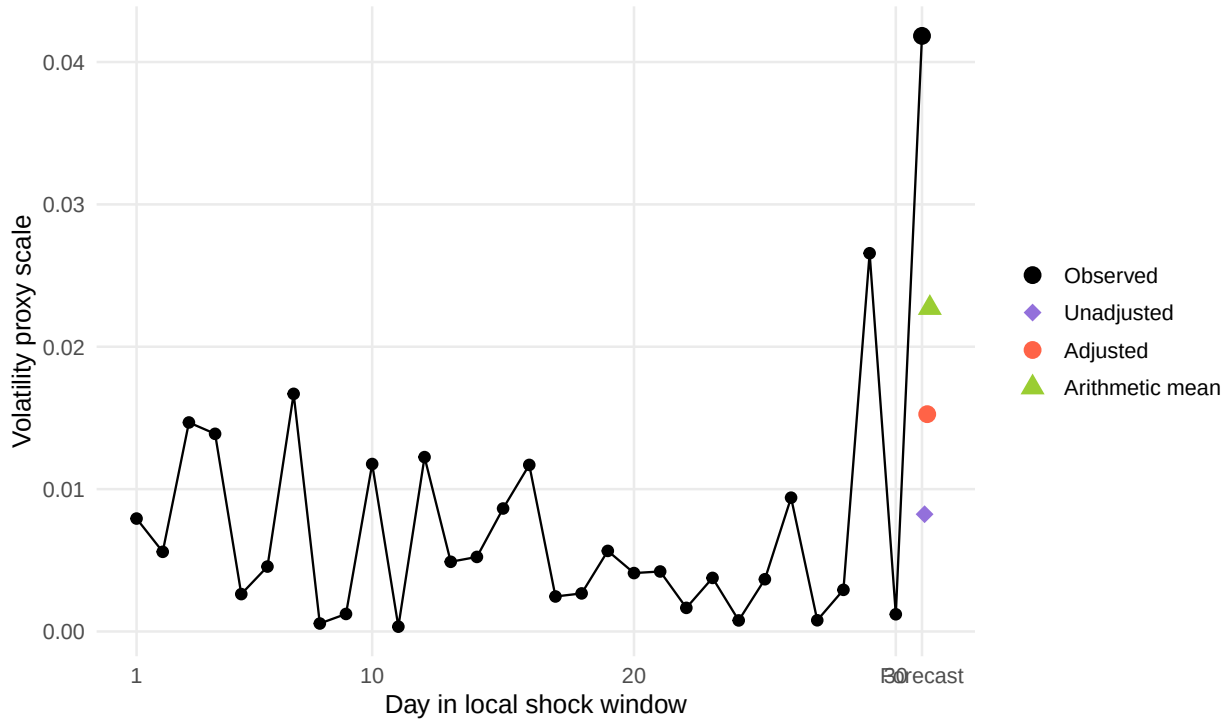
## [1] 0.0005164128

target_reaction_proxy

## [1] 0.001750259

```

IYG local volatility trend and forecast-day comparison (elections only)



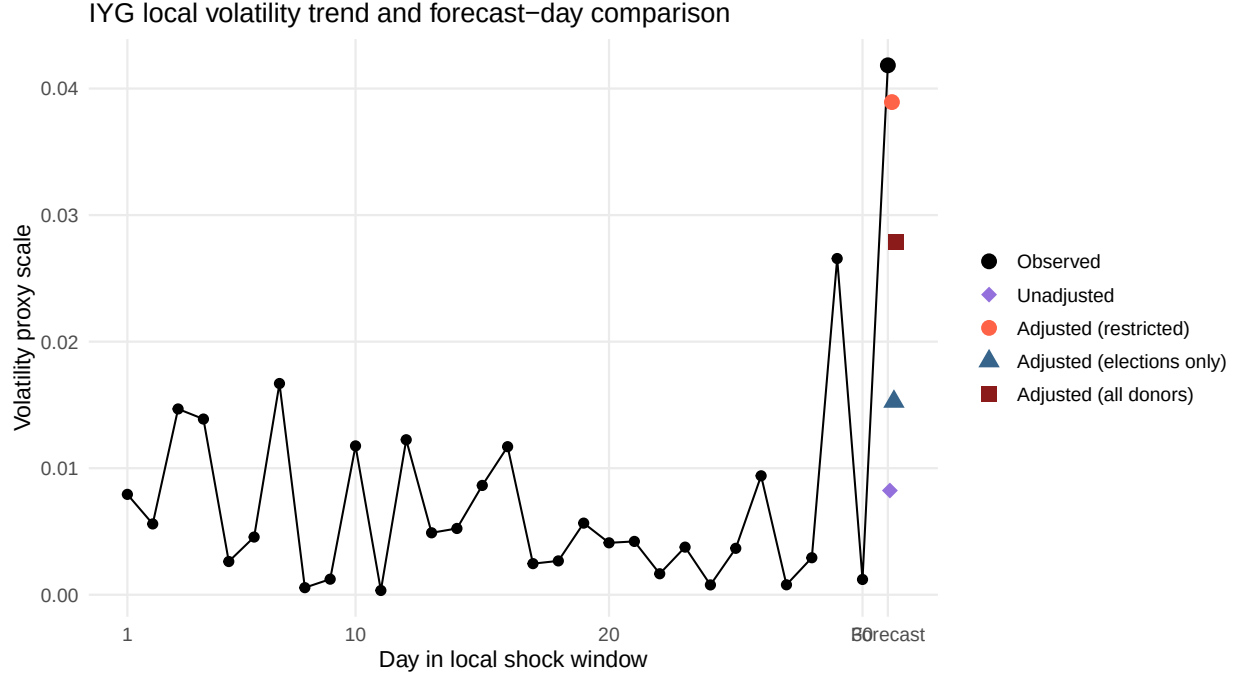
Under the elections-only donor-pool specification, the software returns a donor-adjusted one-step-ahead volatility forecast that is above the no-donor baseline forecast but remains well below the observed next-day variance proxy. In this episode, the donor-adjusted variance forecast is approximately 0.000233, compared with a no-donor baseline forecast of about 0.000068 and an arithmetic-mean benchmark of about 0.000516, while the observed next-day variance proxy is about 0.001750. This indicates that, even under an elections-only donor pool, donor information still implies a positive short-horizon post-shock volatility uplift beyond the target-side pre-shock forecast. However, the magnitude of this uplift is limited, suggesting that same-type election donors alone may not be sufficient to generate a strong adjustment for the 2016 Election episode.

3.3 Comparison Across Donor-Pool Specifications

To assess the role of donor-pool composition in the real-data experiment, we compare three analyst-specified donor-pool constructions for forecasting the one-step-ahead post-shock volatility following the 2016 Election. The first specification uses all available donor episodes in the candidate pool. The second specification uses a restricted donor pool consisting of three economically comparable events: the 2012 Election, Brexit Poll Released, and Brexit. The third specification uses an elections-only donor pool consisting of past U.S. presidential elections: the 2004 Election, 2008 Election, and 2012 Election. For each donor-pool choice, the software quickly returns the corresponding target-only baseline forecast (“unadjusted”), donor-adjusted forecast, and arithmetic-mean benchmark, allowing outputs across competing donor pools to be compared within a common framework. Because the true conditional variance is latent in real data, evaluation is based on the next-day squared return, which serves as an observed proxy for one-step-ahead variance.

Table 1: Comparison of donor-pool-specific one-step post-shock variance forecasts.

Specification	Adjusted	Arith. mean	AE Adj.	AE Mean
Restricted	0.001516	0.001310	0.000235	0.000440
Elections only	0.000233	0.000516	0.001517	0.001234
All donors	0.000778	0.000897	0.000972	0.000853



Taken together, these results suggest that donor-pool composition matters primarily through the magnitude of the implied post-shock volatility uplift rather than its direction. All three donor-pool specifications yield forecasts that are substantially above the no-donor baseline, indicating that donor information consistently contributes to short-horizon post-shock volatility forecasting. At the same time, the results also suggest that a broader donor pool is not automatically preferable, and that donor selection cannot rely on event-type similarity alone. The full donor pool may dilute the adjustment by incorporating less comparable episodes, while the elections-only pool shows that using only same-type historical events does not necessarily generate a strong donor correction. Overall, this case is more consistent with the view that donor selection should balance historical comparability, economic-environment similarity, and pre-shock feature similarity, rather than relying on any single criterion in isolation.

In this sense, the software is useful not only because it quickly returns forecasts once a donor pool is specified, but also because it allows the analyst to evaluate how different substantive donor-pool choices affect the resulting uplift adjustment.